ENTREGA 5: PERSISTÊNCIA DE ESTADO, HISTÓRICO E PRESENÇA

Sensor de Alarme com Detector de Movimento (PIR)

Eduardo Octavio de Paula Souza

1. INTRODUÇÃO

Esta entrega adiciona ao gateway Node.js a persistência dos dados do sistema de alarme em um banco Redis. Até a entrega anterior, o gateway apenas recebia as mensagens MQTT, validava e exibia no terminal. Agora, cada mensagem válida passa a ser gravada, permitindo consultar o último estado do dispositivo, o histórico de leituras e a última vez em que ele foi visto (presença).

2. AMBIENTE DE DESENVOLVIMENTO

• Node.js com as bibliotecas mqtt e redis (versão 4 ou superior);

• Broker Mosquitto executando em Docker (porta 1883);

• Redis 7 executando em Docker (porta 6379), com persistência em disco (appendonly) e volume redis_data;

• ESP32 com sensor PIR, publicando no tópico alarme/sensor/pir.

O Redis foi adicionado ao mesmo docker-compose.yml do Mosquitto:

```
redis:
  image: redis:7
  container_name: redis-iot
  ports:
    - "6379:6379"
  command: redis-server --appendonly yes
  volumes:
    - redis_data:/data
```

3. ESTRUTURA DE CHAVES

As chaves seguem o padrão device:<deviceId>:<tipo>. O deviceId faz parte do nome da chave, o que permite adicionar novos dispositivos sem alterar o código e sem misturar os dados.

```
device:<deviceId>:state     String  último JSON recebido do dispositivo
device:<deviceId>:history   List    últimos 100 registros (mais novo primeiro)
device:<deviceId>:lastSeen  String  data/hora ISO (UTC) da última mensagem
```

O histórico é uma lista na qual cada nova leitura é inserida no início (LPUSH). Em seguida o comando LTRIM mantém apenas os 100 registros mais recentes, evitando o crescimento indefinido da memória.

4. PERSISTÊNCIA DOS DADOS

Somente mensagens que passam na validação (deviceId string, sensorPIR booleano e timestamp numérico) são gravadas. Mensagens inválidas continuam sendo apenas registradas no log como INVÁLIDA e não alteram o Redis.

O campo timestamp enviado pelo ESP32 é o valor de millis(), ou seja, o tempo desde que a placa ligou, e não uma data real. Por isso o gateway adiciona o campo receivedAt, com a data e hora UTC do momento do recebimento, e usa esse valor também em lastSeen. Para exibição, a conversão para o horário de Brasília é feita no momento da consulta.

A cada mensagem válida o gateway executa: SET em state, LPUSH e LTRIM em history, e SET em lastSeen.

5. CÓDIGO-FONTE

Função de persistência do gateway (gateway/index.js):

```
const mqtt = require('mqtt');
const { createClient } = require('redis');

const BROKER_URL = 'mqtt://192.168.2.168:1883';
const REDIS_URL = 'redis://localhost:6379';
const TOPICO_TELEMETRIA = 'alarme/sensor/pir';
const TOPICO_COMANDO = 'alarme/comando/sistema';
const LIMITE_HISTORICO = 100;

const redis = createClient({ url: REDIS_URL });
redis.on('error', (err) => console.log('Erro Redis:', err.message));

function chave(deviceId, tipo) {
  return `device:${deviceId}:${tipo}`;
}

async function persistir(dados) {
  const agora = new Date().toISOString();
  const registro = JSON.stringify({ ...dados, receivedAt: agora });

  await redis.set(chave(dados.deviceId, 'state'), registro);
  await redis.lPush(chave(dados.deviceId, 'history'), registro);
  await redis.lTrim(chave(dados.deviceId, 'history'), 0, LIMITE_HISTORICO - 1);
```

```
  await redis.set(chave(dados.deviceId, 'lastSeen'), agora);

  console.log(`[REDIS] dados de ${dados.deviceId} gravados.`);
}
```

Script de consulta (gateway/consulta.js), que lê o estado, a presença e os 10 últimos registros formatando a hora para America/Sao_Paulo:

```
const { createClient } = require('redis');
const redis = createClient({ url: 'redis://localhost:6379' });
const deviceId = process.argv[2] || 'esp32-alarme-01';

function formatarHora(iso) {
  return new Date(iso).toLocaleString('pt-BR', { timeZone: 'America/Sao_Paulo' });
}

async function main() {
  await redis.connect();
  const state = await redis.get(`device:${deviceId}:state`);
  const lastSeen = await redis.get(`device:${deviceId}:lastSeen`);
  const history = await redis.lRange(`device:${deviceId}:history`, 0, 9);
  // ... impressão formatada do estado, da presença e dos 10 últimos registros
  await redis.quit();
}
main();
```

6. CONSULTAS

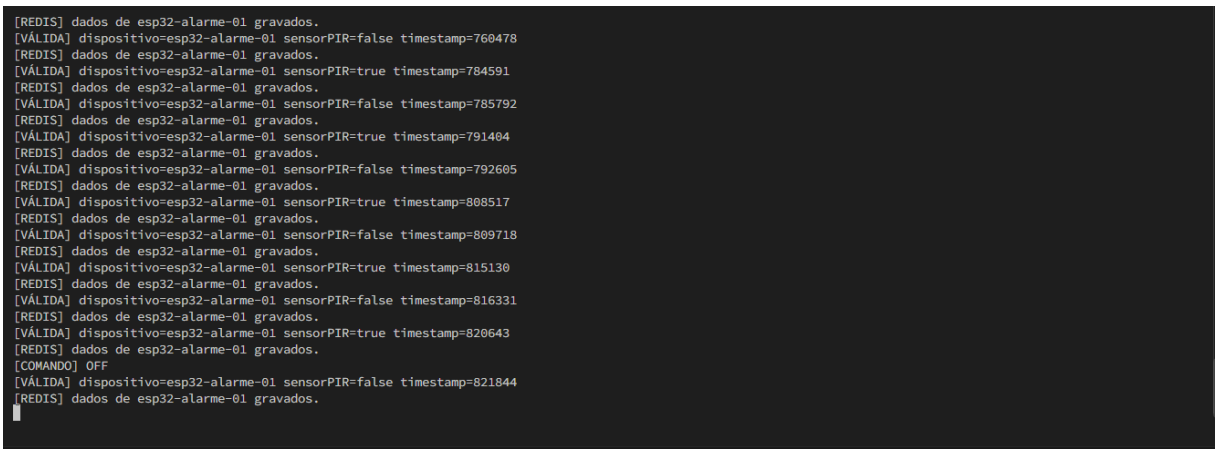
Os dados podem ser consultados pelo redis-cli dentro do container ou pelo script consulta.js:

```
docker exec -it redis-iot redis-cli
KEYS device:*
GET device:esp32-alarme-01:state
GET device:esp32-alarme-01:lastSeen
LRANGE device:esp32-alarme-01:history 0 9
LLEN device:esp32-alarme-01:history

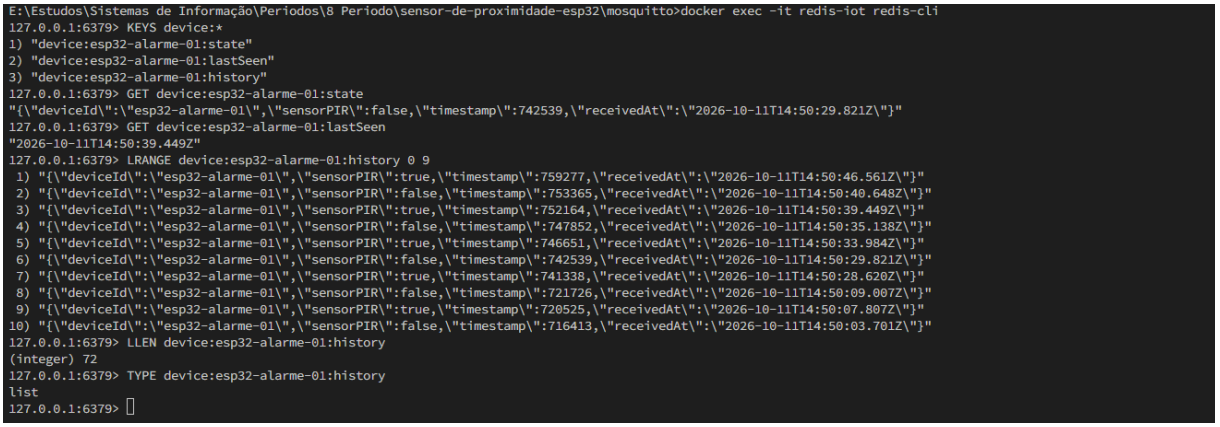
node consulta.js
```

7. EVIDÊNCIA DE FUNCIONAMENTO

Gateway recebendo mensagens do ESP32 e gravando no Redis (linhas [VÁLIDA] e [REDIS]):



Chaves criadas no Redis e conteúdo do estado atual:



Consulta pelo script:



8. CONSIDERAÇÕES FINAIS

A entrega permitiu integrar o gateway ao Redis e definir um modelo de chaves simples, organizado por dispositivo. O estado atual, o histórico limitado e a presença ficam disponíveis para a futura API e para o sistema web. Como o ESP32 só publica quando o estado do PIR muda, o lastSeen é atualizado apenas nessas mudanças; para detectar dispositivo offline de forma mais confiável, seria possível adicionar um envio periódico (heartbeat) ou usar LWT. Além disso, o ESP32 não tem relógio real, por isso o horário de recebimento é definido pelo gateway.